Testing Report

Presented By

ZoltanFrenyo, El Messaoudi Yasmine &
Vaishnu Balakrishnan Sugumaran

Introduction

This report documents the testing of the **Python Network Scanner** tool. The project is modular, with each module responsible for specific functionality. Testing ensures the tool correctly identifies **open and closed ports**, handles **edge cases**, and stores results reliably. All testing was done in a **controlled lab environment**.

Modules Tested

Module	Responsibility
portscanner.py	Main module coordinating scanning and CLI arguments
take_input.py*	Handles interactive user input (IP address, port range)
validate_input.py	Validates user inputs: ensures correct IP addresses, port ranges, and output filenames.
socket_connection.py	Attempts TCP connections to individual ports
scan_iterator.py	Iterates through IP addresses and ports for scanning.
write_to_console.py	Displays results in a formatted console table
store_to_file.py	Saves scan results to a JSON file for record keeping.

***Note :** The take_input.py module was used in an earlier interactive version of the scanner. After implementing the CLI interface using `argparse`, the interactive mode was removed to simplify input handling and avoid untested edge cases. The module was kept in the project to document the earlier design.

Testing Environment

- **Target :** Isolated virtual machine with known open and closed ports.
- **Scanner Host :** Local machine running Python 3.10+
- **Network :** Internal Virtual internal network (lab environment only).
- **Monitoring Tools :** Wireshark used to observe TCP connection attempts.
- **Safety :** Only a controlled lab system was scanned. No external systems were targeted.

Test Cases and Results

Test Case	Module(s)	Expected Result	Actual Result	Status
Valid IP & port input	validate_input.py	Accept input and proceed	✅ Passed	Pass

Invalid IP format	validate_input.py	Reject input and display error	✅ Correct error message	Pass
Invalid port format or range	validate_input.py	Reject invalid ports	✅ Correct error message	Pass
Scan open port (e.g., 22 SSH)	socket_connection.py, scan_iterator.py	Port reported OPEN	✅ OPEN	Pass
Scan closed port (e.g., 23 Telnet)	socket_connection.py, scan_iterator.py	Port reported CLOSED	✅ CLOSED	Pass
Sequential port scan	scan_iterator.py	All ports iterated correctly	✅ Correct iteration	Pass
Console output	write_to_console.py	Display formatted results	✅ Formatted output shown	Pass
Store to file	store_to_file.py	Save results with correct structure	✅ File created correctly	Pass
Wrong file name or type	validate_input.py	Rejects input and displays error	✅ No File created	Pass
Edge case : unreachable host	socket_connection.py	Handle timeout without crashing	✅ Handled	Pass
Edge case : high port numbers	scan_iterator.py, socket_connection.py	Scan ports 49152–65535 accurately	✅ Correct	Pass

Screenshots

```
└─$ python portscanner.py -t 127.0.0.1 -p 80
2026-03-13 14:11:44,588 - INFO - Starting port scan on 127.0.0.1 ports 80-80 with max threads: 100, max connections/sec: 100.0
2026-03-13 14:11:44,589 - INFO - Port scan finished in 0.00134 seconds

Scan Results
──────────────────────────────────────────────────────────────
IP              PORT      STATUS    COMMON PORT
──────────────────────────────────────────────────────────────
127.0.0.1       80        CLOSED    HTTP

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 127.0.0.1 -p 20-25
2026-03-13 14:11:52,413 - INFO - Starting port scan on 127.0.0.1 ports 20-25 with max threads: 100, max connections/sec: 100.0
2026-03-13 14:11:52,465 - INFO - Port scan finished in 0.05209 seconds

Scan Results
──────────────────────────────────────────────────────────────
IP              PORT      STATUS    COMMON PORT
──────────────────────────────────────────────────────────────
127.0.0.1       20        CLOSED    FTP
127.0.0.1       21        CLOSED    FTP
127.0.0.1       22        CLOSED    SSH
127.0.0.1       23        CLOSED    Telnet
127.0.0.1       24        CLOSED    No
127.0.0.1       25        CLOSED    SMTP

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 256.1.1.1 -p 80
IP ERROR: Octets must be between 0 and 255!

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 192.168.0 -p 80
IP ERROR: IP address must have 4 octets!

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 127.0.0.1 -p 70000
PORT ERROR: Port cannot be greater than 65535!

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 127.0.0.1 -p abc
PORT ERROR: Port must be a number between 0 and 65535!

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 127.0.0.1 -p 80 -o test.hello
FILE ERROR: Output file must end with .json

┌──(yasmine㉿kali)-[~/Documents/PortScannerSafety]
└─$ python portscanner.py -t 127.0.0.1 -p 80 -o inv@lid.json
FILE ERROR: Filename contains invalid characters
```

Observations

- **Open ports** were correctly identified by the scanner.
- **Closed ports** were accurately reported.
- **Invalid inputs** were rejected, preventing crashes.
- **Sequential scanning** of multiple ports on the target IP worked correctly**.**
- **Console and file outputs** matched each other.
- Wireshark confirmed the tool initiated **TCP connection attempts (TCP handshake)** to the target ports.
- The tool handled **timeouts and unreachable hosts gracefully**.

- The scanner implements **rate limiting** to control the number of connection attempts per second, preventing excessive network traffic.

Edge Case Notes

- Ports outside **1–65535** are rejected with a validation error.
- Scanning an offline VM results in **timeouts**, but the scanner continues without crashing.
- Sequential port scanning up to **high-numbered ports** works correctly.
- Port scanning performance is improved using multithreading via ThreadPoolExecutor, allowing multiple ports to be scanned concurrently.

Conclusion

The **Python Network Scanner** performs as expected:

- Correctly identifies **open and closed ports**
- Validates user input and prevents invalid scans
- Handles **edge cases** like unreachable hosts or invalid ports
- Outputs results both **to console** and **to file**
- Works reliably within a **controlled lab environment**

All modules **— port_scanner, take_input, validate_input, socket_connection, scan_iterator, write_to_console, store_to_file** — passed their respective tests.

Testing confirms the scanner is **functional, safe, and reliable** for educational and lab use.